

SEMINAR · GIT & GITHUB · FOR VIBE CODERS

바이브 코딩을 위한 최소한의 Git & GitHub

명령어를 외우지 않습니다.

에이전트에게 말로 시킵니다.

사내 GitHub(**github.company.net**) 사용법부터

내가 만든 웹앱을 URL로 공유하는 것까지 — 90분.

오늘 배울 조작법의 전부

복사

지금까지 작업한 것 커밋하고,
사내 깃헙에 올려줘.

← 이 한 문장이면 됩니다. 진짜로.

오늘의 여정 · 이론 30% + 실습 70%

오늘 가져가실 세 가지

개념 딱 6개

Git이 뭔지, 왜 쓰는지.

저장소·커밋·브랜치·푸시·풀·PR — 이게 전부입니다.

CH 1 · 이론

사내 GitHub 사용법

github.com과 **github.company.net**은 뭐가 다른지, 어디에 뭘 올려야 하는지.

CH 2 · 이론 + 실습 5개

내 웹앱 배포

바이브 코딩으로 만든 앱을 GitHub Pages로 — 팀에 URL 한 줄로 공유.

CH 3 · 이론 + 실습 2개

모든 사용법은 프롬프트(말로 시키기)가 기본, 수동 명령어는 아래에 작게, 참고용으로만 소개합니다.

이 폴더... 익숙하지 않으세요?

 보고서_v1.pptx 3월 2일

 보고서_최종.pptx 3월 9일

 보고서_진짜최종.pptx 3월 10일

 보고서_진짜최종_수정.pptx 3월 10일 밤 11시

 보고서_진짜최종_수정2_이걸로.pptx 3월 11일 새벽 2시

Git은 이 비극을 끝냅니다

파일은 하나. 대신 모든 변경의 역사가 자동으로 기록됩니다.

"3월 9일 버전으로 돌아가줘"가 언제든지 가능합니다.

Git = 문서·코드의 타임머신 + 자동 기록장치.

코드만이 아니라 폴더 안 모든 파일에 적용됩니다.

바이브 코딩일수록 **세이브 포인트**가 필요합니다

게임처럼 저장

보스전 전에 세이브하듯, 잘 되는 순간마다 커밋.
AI가 다음 수정에서 망쳐도 안심.

3초 되감기

“아까 잘 되던 걸로 돌려줘” 한 마디면 복구됩니다.
실험이 두렵지 않게 됩니다.

에이전트가 Git 달인

AI 코딩 에이전트는 git을 가장 잘 다루는 도구로
훈련돼 있습니다. 여러분은 시키기만 하면 됩니다.

커밋은 자주, 후회는 없게. — AI가 코드를 빨리 만들수록, 되돌아갈 지점도 촘촘해야 합니다.

오늘 필요한 단어는 6개뿐

1  저장소 repository

프로젝트 폴더 + 그 폴더의 모든 역사

2  커밋 commit

세이프 포인트 - "지금 상태를 기록해줘"

3  브랜치 branch

본편을 건드리지 않는 평행 우주 작업본

4  푸시 push

내 기록을 GitHub(클라우드)에 올리기

5  풀 pull

GitHub의 최신 내용을 내 컴퓨터로 받기

6  PR pull request

"내 작업을 본편에 합쳐주세요" + 동료 리뷰

이 6개면 오늘 내용 전부를 이해할 수 있습니다. 나머지 수백 개의 git 용어는? — 에이전트가 압니다. 우리가 알 필요 없습니다.

Git과 GitHub는 다른 것입니다

Git — 내 컴퓨터의 기록 엔진

내 폴더의 스냅샷(커밋)을 찍고, 되돌리고, 브랜치를 만드는 프로그램.

인터넷 없어도 동작합니다.

GitHub — 그 기록을 올려두는 클라우드

저장소를 보관·공유하고, 동료와 리뷰(PR)하고, 웹사이트(Pages)까지 만들어 주는 서비스(웹사이트).

사진(Git)과 인스타그램(GitHub)의 관계.

 Git
내 컴퓨터의 기록

+

 GitHub
공유 · 협업 · 배포

=

 팀의 타임머신
함께 쓰는 역사

우리에겐 GitHub가 두 개 있습니다

github.com

전 세계 공개 클라우드 — 인터넷의 GitHub

- 개인 이메일로 가입한 개인 계정
- 오픈소스, 개인 공부, 토이 프로젝트
- 무료 공개 저장소는 **전 세계 누구나** 볼 수 있음

github.company.net

회사가 직접 운영하는 GitHub Enterprise

- 회사 계정(사내 SSO)으로 로그인
- 업무 코드·문서는 **반드시 여기**
- 사내망(VPN)에서만 접속 — 밖에서는 안 보임

 회사 코드·데이터는 절대 github.com에 올리지 않습니다. 정보 유출이자 보안 규정 위반입니다.

화면 생김새와 기능은 거의 똑같습니다. 다른 것은 주소, 계정, 그리고 “누가 볼 수 있는가”입니다.

어디에 올리지? — 3초 판단법

Q1 업무 관련 코드·데이터·문서가 조금이라도 들어있다

→ 🏢 사내 GitHub

Q2 회사 정보 0%인 개인 공부·취미 프로젝트

→ 🌐 github.com (개인 계정)

Q3 애매하다, 잘 모르겠다

→ 🏢 사내 GitHub (안전한 쪽)

💡 습관 하나

업무용 브라우저에는 **github.company.net**만 로그인해두기. 계정이 섞이면 실수가 시작됩니다.

🔍 에이전트에게도 알려주기

“이 프로젝트의 원격 저장소는 사내 GitHub야”라고 한 번 말해두면, 에이전트가 주소를 헛갈릴 일이 없습니다.

오늘의 조작법 — 프롬프트가 곧 명령어

Claude Code, OpenCode 같은 코딩 에이전트는 git을 훈련받은 전문가입니다.
앞으로 나올 모든 실습 슬라이드는 이 구조입니다:

💬 이렇게 말하세요 - 기본 조작법 (크게)

복사

한국어로, 하고 싶은 일을 그대로. 정확한 용어가 아니어도 됩니다.

🔑 수동으로 하려면 - 참고용 (작게)

\$ 터미널에 직접 치는 명령어. 몰라도 됩니다. 궁금할 때만 보세요.

“왜 명령어부터 안 배우나요?” — 운전을 배울 때 엔진 구조부터 배우지 않는 것과 같습니다. 필요해지는 날, 그때 배워도 늦지 않습니다.

👉 실습 0 · 준비물 점검 (5분)

시작 전 3가지만 확인합니다

1

에이전트 플랫폼

Claude Code / OpenCode 등 — 어느 것이든 오늘 내용은 동일합니다



2

사내 GitHub 로그인

github.company.net 이 브라우저에서 열리는지



3

실습 폴더

바이브 코딩으로 만든 앱 폴더 (없으면 빈 폴더 + 간단한 index.html)

💬 에이전트에게 이렇게 말하세요

복사

git이 설치돼 있는지 확인해줘. 없으면 설치해주고,
커밋 기록에 쓸 이름과 회사 이메일을 설정해줘.

🔧 수동으로 하려면

```
git --version
git config --global user.name "홍길동"
git config --global user.email "hong@company.com"
```

👉 실습 1 · 첫 세이프 포인트 (5분)

내 폴더를 저장소로 만들고, 첫 커밋

💬 에이전트에게 이렇게 말하세요

복사

이 폴더를 git 저장소로 만들고, 지금까지의 작업 전체를
"첫 버전"이라는 메시지로 커밋해줘.

✅ 방금 일어난 일

폴더 안에 숨겨진 `.git`(역사 보관함)이 생기고, 세이프 포인트 1호가 기록됐
습니다.

💡 커밋 메시지도 말기세요

"적당한 메시지로 커밋해줘"라고 하면 에이전트가 변경 내용을 요약해서 써 주
니다.

🔧 수동으로 하려면

```
git init && git add -A && git commit -m "첫 버전"
```

내 컴퓨터 밖으로 — push

1

웹에서 저장소 만들기

github.company.net 접속 → New repository → 이름
(예: my-first-app)만 정하고 Create



2

에이전트로 연결 + 푸시

아래 프롬프트 (주소는 만든 저장소 페이지에서 복사)



3

확인

브라우저 새로고침 — 내 파일이 보이면 성공 🎉

💬 에이전트에게 이렇게 말하세요

복사

사내 깃헙에 저장소를 만들었어:

https://github.company.net/내아이디/my-first-app

여기를 원격(remote)으로 연결하고 지금까지 커밋을 푸시해줘.

🔑 수동으로 하려면

```
git remote add origin https://github.company.net/내아이디/my-first-app.git
git push -u origin main
```

푸시할 때 로그인/토큰을 물어보면 → 에이전트에게 “사내 깃헙 토큰 발급 방법 알려줘”라고 물어보세요. (Settings → Developer settings → Personal access tokens)

👉 실습 3 · 일상 루프 (5분)

매일 반복될 리듬: 수정 → 커밋 → 푸시

STEP 1

🎨 바ιβ 코딩

에이전트와 기능 수정

STEP 2

👁️ 확인

브라우저에서 잘 되는지 내 눈
으로

STEP 3

💾 커밋

"커밋해줘" — 세이브 포인트

STEP 4

⬆️ 푸시

"푸시해줘" — 클라우드 백업
+ 팀 공유

💬 에이전트에게 이렇게 말하세요 (STEP 3+4 한 번에)

복사

지금까지 변경사항을 보기 좋게 커밋하고 푸시해줘.

🔧 수동으로 하려면

```
git add -A && git commit -m "다크모드 추가" && git push
```

커밋 타이밍: 하루의 끝이 아니라 기능 하나가 잘 될 때마다. 실습으로 아무 파일이나 한 줄 고치고 위 프롬프트를 실행해보세요.

망했다? 한 문장이면 복구됩니다

💬 방금 AI가 코드를 망쳤을 때

복사

마지막 커밋 이후의 변경을 전부 버리고, 마지막 커밋 상태로 되돌려줘.

💬 과거의 잘 되던 버전으로 가고 싶을 때

복사

최근 커밋 목록을 보여줘. 어제 오후에 잘 되던 버전으로 돌아가고 싶어.

💬 커밋만 취소하고 파일은 그대로 두고 싶을 때

복사

마지막 커밋을 취소해줘. 파일 내용은 그대로 유지하고.

🔧 수동으로 하려면

```
git log --oneline      # 세이브 포인트 목록
git restore .          # 커밋 안 한 변경 버리기
git revert HEAD        # 마지막 커밋을 "취소하는 커밋" 만들기
```

⚠️ "전부 삭제", "강제로" 같은 요청에는 에이전트가 한 번 더 확인을 요청합니다 — 그 메시지는 꼭 읽고 답하세요.

본편은 안전하게, 실험은 평행 우주에서

🌱 브랜치

main(본편)은 늘 잘 도는 상태로 두고, 새 기능은 복사본 우주에서 실험합니다.

🙏 Pull Request

“이 우주를 본편에 합쳐주세요” 요청. 동료가 웹에서 리뷰하고 버튼 하나로 합칩니다(merge).

혼자 하는 프로젝트라면 당분간 main에 바로 커밋해도 충분합니다. 팀 작업이 시작되는 순간, 이 슬라이드로 돌아오세요.

💬 브랜치 만들어 작업 시작

복사

새 기능은 `feature/dark-mode` 브랜치를 만들어서 거기서 작업해줘.

💬 작업 끝 - PR 만들기

복사

작업 끝났어. 푸시하고 `main`으로 Pull Request를 만들어줘. 제목과 설명도 채워줘.

🔧 수동으로 하려면

```
git checkout -b feature/dark-mode
# 작업 후 push → 웹에서 "Compare & pull request" 버튼
```

자주 만나는 상황 — 이렇게 말하세요

✂️ “충돌(conflict)이 났다는데요?”

💬 충돌 났어. 안전하게 해결해줘. 판단이 애매한 부분은 나한테 물어봐.

🧐 “뭐가 바뀌었는지 모르겠어요”

💬 어제 이후 변경사항을 쉬운 말로 요약해줘.

🤝 “동료 최신 코드를 받고 싶어요”

💬 원격 저장소의 최신 내용을 받아와줘(pull). 내 작업이랑 겹치면 알려줘.

🚫 “푸시가 거부됐어요”

💬 푸시가 거부됐어. 원인 확인하고 안전하게 해결해줘.

📁 “동료 저장소를 통째로 받고 싶어요”

💬 이 주소를 클론해줘: <https://github.com/company.net/팀/저장소>

🔑 “API 키를 실수로 커밋했어요”

💬 방금 커밋에 시크릿이 들어갔어. 파일과 기록에서 제거해줘. + 키는 즉시 재발급!

공통 패턴: 상황을 그대로 설명하면 됩니다. 정확한 용어를 몰라도, 에러 메시지를 복사해서 붙여넣어도 됩니다.

저장소가 곧 웹사이트가 됩니다 — GitHub Pages

1

내 저장소

바이브 코딩으로 만든 html / css / js 파일들



2

Pages 스위치 ON

저장소 설정에서 클릭 몇 번 (다음 슬라이드)



3

URL 완성

서버 없이, 무료로, 누구나 접속



github.com이라면

<https://아이디.github.io/저장소/>



사내 GitHub라면

<https://pages.company.net/아이디/저장소/>

회사 설정에 따라 <https://github.company.net/pages/...> 형태일 수도 있습니다.

바이브 코딩으로 만든 계산기, 팀 대시보드, 소개 페이지... “이거 봐봐” 한 줄 URL로 공유하세요.

배포 방식은 두 가지 — 오늘은 **branch** 방식만

GitHub Actions 방식

푸시하면 로봇이 자동으로 빌드·배포. 강력하지만 사내에서는 러너 설정·권한 절차가 까다롭습니다.

오늘은 다루지 않음

Branch 방식 (Deploy from a branch)

“이 브랜치의 파일을 그대로 웹으로 내보내줘”라고 GitHub에 알려주는 것. 설정 파일도, 서버 지식도 필요 없습니다.

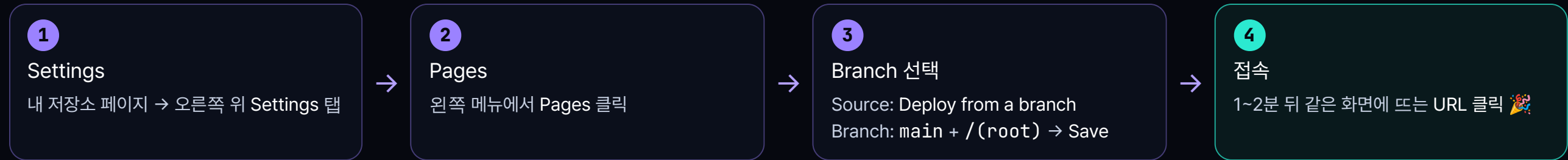
오늘의 방법 — 클릭 4번

단순한 방법이 충분히 좋은 방법입니다. *branch* 방식으로 안 되는 게 생기는 날 — 그때 *Actions*를 배우면 됩니다.

👉 실습 6 · 첫 배포 - 정적 앱 (10분)

클릭 4번으로 내 앱을 웹에 올립니다

전제: 실습 2에서 푸시한 저장소에 `index.html`이 있습니다. Pages 설정은 웹에서 클릭이 기본입니다:



💬 배포 전에 에이전트에게 점검을 맡기세요

복사

이 앱을 GitHub Pages(branch 방식)에 올릴 거야.
하위 경로에서 열려도 깨지지 않게 절대 경로 같은 문제가 있는지
점검해서 고쳐줘. 그리고 커밋, 푸시까지 해줘.

💡 `docs/` 폴더만 공개하는 옵션(Branch: main + /docs)도 있습니다 — 저장소 전체가 아니라 일부만 웹에 내보내고 싶을 때.

React·Vite 앱은 빌드 결과물을 올립니다

React/Vue/Vite 앱은 소스 그대로는 웹에서 안 돌아가고, 빌드된 산출물(dist 폴더)만 올려야 합니다. Actions 없이 하는 요령: **dist**를 gh-pages 브랜치에 밀어 넣고, Pages가 그 브랜치를 보게 하는 것.

💬 에이전트에게 이렇게 말하세요 (전부 말하기)

복사

이 앱을 GitHub Pages에 branch 방식으로 배포하고 싶어.

- 1) base 경로를 /저장소이름/ 으로 설정하고
- 2) 빌드해서 결과물(dist)을 gh-pages 브랜치로 푸시해줘.
- 3) 내가 Pages 설정에서 뭘 선택해야 하는지 알려줘.

🔑 수동으로 하려면

```
npm run build
git subtree push --prefix dist origin gh-pages
# Pages 설정: Branch를 gh-pages + /(root)로
```

💬 다음부터 업데이트할 때

복사

수정한 내용으로 다시 배포해줘.

💬 새로고침하면 404가 뜰 때 (SPA 라우팅)

복사

Pages에서 새로고침하면 404가 떠.
해시 라우팅으로 바꾸거나 404.html 우회를 적용해줘.

배포가 이상할 때 — 체크리스트

☐ “화면이 하얗게만 나와요”

💬 배포한 페이지가 하얗게 나와. 브라우저 콘솔 에러 기준으로 경로 문제 고쳐줘.

 “404 Not Found가 떠요”

💬 Pages 설정의 브랜치/폴더가 맞는지, index.html이 그 위치에 있는지 확인해줘.

 “반영이 안 돼요”

💬 첫 배포·업데이트 반영은 몇 분 걸립니다 + 강력 새로고침 Ctrl+Shift+R

 “동료가 링크를 못 열어요” (사내)

💬 사내 Pages는 저장소 접근 권한을 따릅니다 — 저장소를 팀에 공개(Internal)하거나 동료를 초대해줘.

디버깅의 제1원칙: 보이는 그대로 에이전트에게 말하기. 스크린샷·에러 메시지를 붙여넣으면 대부분 해결됩니다.

⚠ 꼭 지켜야 할 것

보안 수칙 4가지 — 이것만은 외우세요

1 회사 코드는 사내 GitHub에만

github.com 업로드 = 외부 유출. 주소창을 확인하는 습관:

github.company.net

2 비밀번호·API 키는 커밋 금지

키는 .env 파일에 두고 .gitignore로 제외.

💬 "커밋하기 전에 시크릿이 들어있는지 검사해줘"

3 이미 새 나갔다면 — 재발급이 1순위

기록에서 지우는 것보다 키를 즉시 무효화·재발급하는 게 먼저입니다. 한 번 푸시 된 것은 "본 사람이 있다"고 가정하세요.

4 Pages는 "게시"라는 뜻

사내 Pages도 설정에 따라 전 임직원이 볼 수 있습니다. 올리기 전에 내용(고객 정보, 미공개 수치)을 한 번 점검.

이 장표만 찍어 가세요 📸

시작 💬 이 폴더를 git 저장소로 만들고 “첫 버전”으로 커밋해줘

일상 저장 💬 변경사항 보기 좋게 커밋하고 푸시해줘

되돌리기 💬 마지막 커밋 상태로 되돌려줘 / 커밋 목록 보여줘

복제 💬 이 저장소를 클론해줘: [https://github.company.net/...](https://github.company.net/)

PR 💬 푸시하고 main으로 Pull Request 만들어줘. 설명도 채워줘

배포(정적) 💬 Pages(branch 방식)에 올릴 거야. 경로 점검하고 커밋·푸시해줘

보안 💬 커밋 전에 시크릿(키·비밀번호) 들어있는지 검사해줘

연결+업로드 💬 이 주소를 원격으로 연결하고 푸시해줘: [https://github.company.net/...](https://github.company.net/)

상태 확인 💬 지금 커밋 안 된 변경이 뭐가 있는지 요약해줘

받아오기 💬 원격의 최신 내용을 받아와줘 (pull)

브랜치 💬 feature/○○ 브랜치 만들어서 거기서 작업해줘

충돌 💬 충돌 났어. 안전하게 해결해줘. 애매하면 물어봐

배포(SPA) 💬 base 경로 /[저장소](#)/ 설정하고 dist를 gh-pages 브랜치로 푸시해줘

배움 💬 방금 네가 한 git 작업을 초보자에게 설명하듯 알려줘

같은 내용이 저장소의 `github/README.md`에도 정리돼 있습니다 — 복사해서 쓰세요.

마무리

기억할 것은 세 문장입니다

1 커밋은 세이브 포인트 — 기능이 잘 될 때마다 “커밋해줘”

2 업무 코드는 `github.company.net` — `github.com`이 아닙니다

3 만든 것은 Pages(branch 방식)로 — URL 한 줄로 공유

질문은 화면 아래 입력창으로 

이 자료는 어느 회사에서나 재사용할 수 있습니다 — 사내 주소 표기는 `github/index.html` 상단의 `SEMINAR_CONFIG`에서 바꾸세요.